

01_tokens

ALUNO: Guilherme Yukio Yamaguchi | RA: 24062624-2

ENTRADA: total_gy = valor + 42;

LEXEMA	TOKEN

total_gy	IDENTIFICADOR
=	ATRIBUICAO
valor	IDENTIFICADOR
+	SOMA
42	INTEIRO
;	PONTO_E_VIRGULA

02_lexico

=== TESTE C2 - identificador personalizado ===

ENTRADA: int nota_gy = 10;

LEXEMA	CLASSE	TOKEN

int	LETRA	PALAVRA_RESERVADA
nota_gy	LETRA	IDENTIFICADOR
=	SIMBOLO	ATRIBUICAO
10	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

=== TESTE D - caractere inválido ===

ENTRADA: int valor# = 1;

LEXEMA	CLASSE	TOKEN

int	LETRA	PALAVRA_RESERVADA
valor	LETRA	IDENTIFICADOR
#	DESCONHECIDO	ERRO_LEXICO
=	SIMBOLO	ATRIBUICAO
1	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

03_erro_compilador

```
public class Main {
    static final String ALUNO = "Guilherme Yukio Yamaguchi";
    static final String RA = "24062624-2";

    public static void main(String[] args) {
        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);

        int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula

        if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
            System.out.println("Resultado: " + total);
        }
    }
}
```

```
Main.java:8: error: ';' expected
        int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e
        virgula
                        ^
Main.java:10: error: ')' expected
        if (total > 20 { // CORRIGIR 2: feche corretamente o
        parenteses
                        ^
Main.java:11: error: unclosed string literal
        System.out.println("Programa corrigido!"); // CORRIGIR
3: feche o texto entre aspas
                        ^
```

04_programa_corrigido

ALUNO: Guilherme Yukio Yamaguchi | RA: 24062624-2

Programa corrigido!

Resultado: 30

05_descendente

ALUNO: Guilherme Yukio Yamaguchi | RA: 24062624-2

ENTRADA: (2 + 3) * 4

```
ENTRANDO em <expr> | lookahead = (
  ENTRANDO em <term> | lookahead = (
    ENTRANDO em <factor> | lookahead = (
      Abrindo parenteses
      Novo lookahead: 2
      ENTRANDO em <expr> | lookahead = 2
        ENTRANDO em <term> | lookahead = 2
          ENTRANDO em <factor> | lookahead = 2
            Numero reconhecido: 2
            Novo lookahead: +
            SAINDO de <factor> | lookahead = +
          SAINDO de <term> | lookahead = +
        Encontrado operador +
        Novo lookahead: 3
        ENTRANDO em <term> | lookahead = 3
          ENTRANDO em <factor> | lookahead = 3
            Numero reconhecido: 3
            Novo lookahead: )
          SAINDO de <factor> | lookahead = )
        SAINDO de <term> | lookahead = )
      SAINDO de <expr> | lookahead = )
    Novo lookahead: *
    Fechando parenteses
    SAINDO de <factor> | lookahead = *
  Encontrado operador *
  Novo lookahead: 4
  ENTRANDO em <factor> | lookahead = 4
    Numero reconhecido: 4
    Novo lookahead: $FIM$
  SAINDO de <factor> | lookahead = $FIM$
SAINDO de <term> | lookahead = $FIM$
SAINDO de <expr> | lookahead = $FIM$
```

06_ascendente

ALUNO: Guilherme Yukio Yamaguchi | RA: 24062624-2

ENTRADA: id+id*id

PILHA	ENTRADA	ACAO

\$ 0	id+id*id\$	SHIFT id ->
estado 5		
\$ 0 id 5	+id*id\$	REDUCE F -> id
\$ 0 F 3	+id*id\$	REDUCE T -> F
\$ 0 T 2	+id*id\$	REDUCE E -> T
\$ 0 E 1	+id*id\$	SHIFT + ->
estado 6		
\$ 0 E 1 + 6	id*id\$	SHIFT id ->
estado 5		
\$ 0 E 1 + 6 id 5	*id\$	REDUCE F -> id
\$ 0 E 1 + 6 F 3	*id\$	REDUCE T -> F
\$ 0 E 1 + 6 T 9	*id\$	SHIFT * ->
estado 7		
\$ 0 E 1 + 6 T 9 * 7	id\$	SHIFT id ->
estado 5		
\$ 0 E 1 + 6 T 9 * 7 id 5	\$	REDUCE F -> id
\$ 0 E 1 + 6 T 9 * 7 F 10	\$	REDUCE T -> T *
F		
\$ 0 E 1 + 6 T 9	\$	REDUCE E -> E +
T		
\$ 0 E 1	\$	ACCEPT